

Context Switching in OS | Medium Term Scheduler | Orphan/Zombie Process

1. Swapping in Operating Systems

When a system is running many programs at once, *main memory (RAM)* may not be sufficient to hold all processes at the same time. To handle such situations efficiently, **swapping** is used — an age-old yet fundamental technique in multitasking operating systems.

What is Swapping?

Swapping is the process of **moving a process out of main memory to secondary storage (disk)** and bringing it back to memory later when needed.

This helps manage limited memory and maintain smooth multitasking.

The main idea is simple:

When RAM becomes full, some processes are temporarily shifted to disk.

Later, when CPU needs them again, they are brought back to RAM.

Role of Medium-Term Scheduler (MTS)

- In **time-sharing systems**, a **Medium-Term Scheduler (MTS)** manages swapping.
- It decides *which process should be swapped out and when to bring it back in*.
- **Swap-out** = Move process from RAM → Disk
- **Swap-in** = Bring process back from Disk → RAM

So, MTS acts like the gatekeeper of memory usage.

Why Swapping is Needed?

Reason	Meaning
Reduce multiprogramming degree	Free memory by temporarily removing processes
Improve process mix	Keep active/interactive processes in memory
Memory overcommitment	When total memory demand > available memory
Maintain system responsiveness	Removes inactive or waiting processes to give space to active ones

Real-World Example

Imagine a computer running 5 applications:

1. Chrome
2. VS Code
3. Spotify
4. Photoshop
5. A game

Your system has **8 GB RAM**, but videoeditor + Game together need more memory. RAM is now overloaded.

What OS does:

1. You start playing game → System prioritizes it.
2. Spotify is running but idle in background → OS **swaps it out to disk**.
3. Game gets the required RAM and runs smoothly.
4. When you return to Spotify window → OS **swaps it back in** to RAM.
5. Spotify resumes music exactly from where it paused — **execution continues from the last saved state**.

- ✓ No loss of progress
- ✓ Efficient use of RAM
- ✓ System remains responsive

Technical Flow of Swapping

Process in RAM → (Swap-Out) → Moved to Disk → Free RAM



Needed again by CPU



Process swapped back → (Swap-In) → Loaded into RAM

Advantages

- Allows more programs to run than main memory can hold.
- Improves utilization of CPU and memory.
- Ensures smooth multitasking even with low RAM.

Disadvantages

- Swap-in/Swap-out takes time → Can slow the system.
- Frequent swapping leads to **thrashing** (system spends more time swapping than executing).
- SSD/HDD wear increases slightly over time.

Final

Swapping ensures efficient memory management by temporarily moving idle processes from RAM to disk. It helps the system run more programs together without slowing down active work. Execution resumes from the exact point when swapped back in — just like pausing and playing a video.

2. Context Switching

Modern operating systems allow multiple processes to share the CPU.

When the CPU shifts from one process to another, the switch must be smooth and precise.

What happens in a Context Switch?

Whenever the CPU is moved from **Process A** → **Process B**, the OS must:

1. **Save the current state of Process A**
(register values, program counter, stack pointer etc.)
2. **Store this saved information inside PCB (Process Control Block) of A**
3. **Load the stored context of Process B** from its PCB
4. CPU then starts executing Process B exactly from the point it was paused.

No progress of either process is lost — it simply *picks up where it left off*.

Why is it called Overhead?

Because during switching:

No actual program/task is executed — CPU only performs saving & restoring work.

So time is consumed, but no useful work for the user is done.

Factors affecting context switch speed

- CPU register count
- Memory access speed
- PCB storage & retrieval time
- Hardware architecture differences

Some systems switch extremely fast, others slower — depends on hardware capability.

Example to Visualize

Two players are using the same computer:

- Player 1 is writing code
- Player 2 is listening to music

CPU runs Player 1 → saves his state → gives CPU to Player 2

After some time → CPU saves Player 2 state → switches back to Player 1

This back-and-forth shift is **context switching**.

3. Orphan Process

Definition

A process becomes **orphan** when:

Its parent finishes or terminates, but
the child process continues running.

What happens to orphans?

They do not remain abandoned.

The operating system assigns them a new parent:

The init process becomes their guardian.

init is the **first process created by OS** at boot time,
and has PID = 1.

Example

1. Parent process forks a child
2. Parent exits or crashes unexpectedly
3. Child is still running → now it has **no parent**

4. OS reassigns child to init

So orphan processes don't harm the system — they simply get adopted.

Orphan Process – Simple Example

A teacher gives homework to a student and then leaves the school early.

But the student is still doing the homework in the classroom.

- The teacher (parent) is **gone**
- The student (child) is **still working**
- Now the student is guided by someone else, like the principal

This is just like an Orphan Process.

In one line:

Parent is gone, but the child is still running. Someone else takes the responsibility.

4. Zombie Process (Defunct Process)

A zombie is a **dead process that is still listed in process table.**

Why does this happen?

When a child process finishes execution:

- It returns an exit status
- But parent must call **wait()** to collect that status

Until parent reads the exit status,
the child's process table entry cannot be removed,
so it stays as a **Zombie**.

How Zombie is Removed? (Reaping)

Zombie state ends when:

Parent calls wait()

↓

Exit status is collected

↓

OS removes child entry from process table

This cleanup process is known as **reaping the zombie**.

Why Zombies Occur? (Example)

1. Parent starts a child process
2. Child finishes in 1 second
3. Parent is busy and calls wait() after 10 seconds

For those 9 seconds, child is:

Terminated but still present in process table → Zombie

Zombie ≠ running

Zombie = just waiting for parent to clean it up.

Zombie Process

A teacher gives homework to a student.

✓ The student finishes the homework.

✗ But the teacher does not check it or mark it as completed.

So what happens?

- The student's work is **done** (like a finished process)
- But the homework is still lying on the table **waiting to be checked**
- That un-checked homework is just like a **Zombie process**

In one line:

Work is finished, but the record is still there. That's a Zombie.